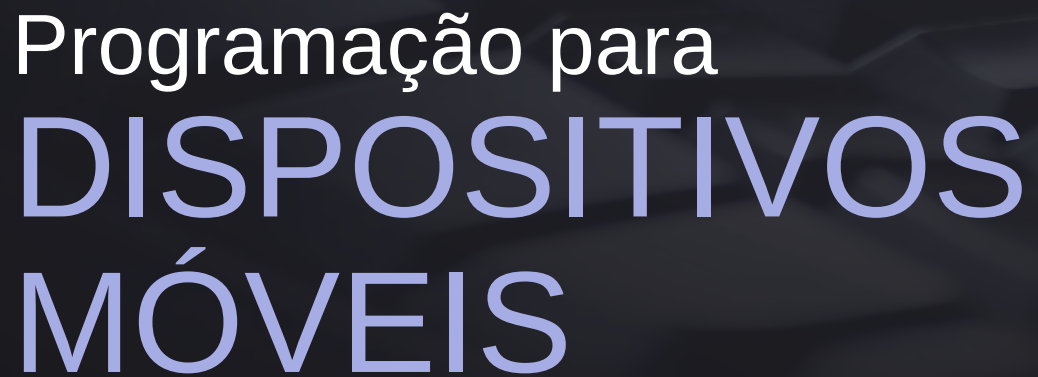




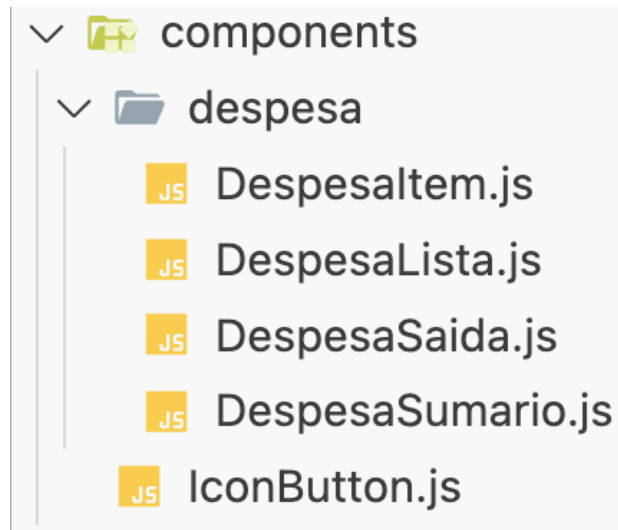
Aula 08: Itens de Interface e Funções

Prof. José Alberto S. Torres



Criação das Telas

- Temos três telas pra criar:
 - TodasDespesas.js
 - GerenciarDespesa.js
 - DespesasRecentes.js
- Para isso, vamos criar quatro componentes adicionais dentro da pasta **components>despesa**:
 - DespesaItem.js
 - DespesaLista.js
 - DespesaSaida.js
 - DespesaSumario.js



Componentes

Ajuste o código das demais telas

```
import {Text} from 'react-native';

function DespesaItem(){

  return (
    <Text>Item </Text>
  )
}

export default DespesaItem;
```

```
import {Text} from 'react-native';

function DespesaSaida(){

  return (
    <Text>Saida</Text>
  )
}

export default DespesaSaida;
```

```
import {Text} from 'react-native';

function DespesaSumario(){

  return (
    <Text>Sumario</Text>
  )
}

export default DespesaSumario;
```

```
import {Text} from 'react-native';

function DespesaLista(){

  return (
    <Text>Lista</Text>
  )
}

export default DespesaLista;
```

Atividade

- Ajuste o seu arquivo DespesaSaida.js para incorporar DespesaSumario.js e DespesaLista.js, de acordo com o apresentado ao lado.
- Adicione como parâmetro de entrada da função principal, uma **propriedade** de nome **despesas**.



Atividade

JS DespesaSaida.js X

```
aula_09 > components > despesa > JS DespesaSaida.js > ...
1  import DespesaSumario from './DespesaSumario';
2  import DespesaLista from './DespesaLista';
3
4  function DespesaSaida({despesas}){
5
6      return (
7          <View>
8              <DespesaSumario />
9              <DespesaLista />
10         </View>
11     )
12
13 }
14
15
16 export default DespesaSaida;
17
```



DespesaSumario.js

- Vamos agora construir nosso componente DespesaSumario.js.
- Ele recebe o **período em que as compras foram realizadas e a lista de compras, e efetua a soma dos valores de cada uma das despesas.**

.reduce()

- Para efetuar a soma, iremos utilizar o método reduce.
- Ele é utilizado para **reduzir um array a um único valor**, aplicando uma função **acumuladora** em cada elemento, um por um.

- A sintaxe é:

```
array.reduce((acumulador, valorAtual) => {  
  // lógica  
  return novoAcumulador;  
}, valorInicial);
```

- Onde:
 - **acumulador**: o resultado acumulado da função.
 - **valorAtual**: o elemento atual do array.
 - **valorInicial** (opcional): valor com o qual o acumulador começa.



Atividade

- Adicione duas props de entrada, em formato desconstruído, em DespesaSumario :
 - uma **lista de despesas** numa prop chamada **despesas**;
 - uma string representando o período numa prop chamada **periodo**.
- Crie uma constante chamada somaDespesas com a soma dos valores do atributo valor da propriedade despesa (despesa.valor), usando reduce.

Atividade

```
import { View, Text } from 'react-native';

function DespesaSumario({despesas, periodo}){
  const somaDespesas = despesas.reduce((total, despesa) => {
    return total + despesa.valor;
  }, 0);

  return (
    <View>

    </View>
  );
}

export default DespesaSumario;
```



Atividade

- No retorno de DespesaSumario, adicione uma View com os seguintes componentes filhos:
 - Um Text que exibe a prop de entrada período
 - Um Text que exibe a constante com a soma dos valores, precedido de R\$.

Atividade

```
import { View, Text } from 'react-native';

function DespesaSumario({despesas, periodo}){
  const somaDespesas = despesas.reduce((total, despesa) => {
    return total + despesa.valor;
  }, 0);

  return (
    <View>
      <Text>{periodo}</Text>
      <Text>R$ {somaDespesas.toFixed(2)}</Text>
    </View>
  );
}

export default DespesaSumario;
```





toFixed()

- O método `.toFixed(n)` **formata um número com exatamente n casas decimais, arredondando se necessário.**

```
let preco = 5.6789;  
console.log(preco.toFixed(2)); // "5.68"
```



DespesaLista.js

- Vamos desenvolver agora a nossa lista de despesas.
- A função principal de DespesaLista **recebe um array com as despesas e lista os itens.**
- Utilizaremos um componente do tipo **FlatList** para exibir a lista de despesas.



FlatList

- É um componente do React Native usado para **renderizar listas grandes de dados** de forma **otimizada**, ou seja, ele carrega apenas os itens visíveis na tela, economizando memória e melhorando a performance.
- **Propriedades Importantes:**
 - data - Array de dados a ser listado
 - renderItem - Função que define **como renderizar** cada item
 - keyExtractor - Função que retorna a **chave única** de cada item



FlatList – Sintaxe

```
import { FlatList, Text, View } from 'react-native';

const dados = [
  { id: '1', nome: 'Maria' },
  { id: '2', nome: 'João' },
];

export default function App() {
  return (
    <FlatList data={dados} keyExtractor={({item}) => item.id} renderItem={({ item }) => (
      <View>
        <Text>{item.nome}</Text>
      </View>
    )} />
  ); }
```

Atividade

JS DespesaLista.js ×

aula_09 > components > despesa > JS DespesaLista.js > ...

```
1  import {Text, FlatList, View} from 'react-native';
2
3  function renderDespesaItem(itemData){
4      return (
5          <View>
6              <Text>{itemData.item.descricao}</Text>
7              <Text>{itemData.item.valor}</Text>
8          </View>
9      )
10 }
11
12
13 function DespesaLista({despesas}){
14
15     return (
16         <FlatList data={despesas}
17             renderItem={renderDespesaItem}
18             keyExtractor={(item) => item.id} />
19     )
20 }
21
22 export default DespesaLista;
```

Ajuste o código do seu
componente
DespesaLista.js para
usar FlatList



Ajustes em DespesaSaida

- Já temos o DespesaSumario.js, DespesaLista.js e DespesaSaida.js com a nossa primeira formatação.
- Agora precisamos adicionar os parâmetros de entrada de DespesaSaida e ajustar os componentes na sua tela correspondente!.



Atividade

JS DespesaSaida.js ×

aula_09 > components > despesa > JS DespesaSaida.js > ...

```
1  import {View} from 'react-native';
2  import DespesaSumario from './DespesaSumario';
3  import DespesaLista from './DespesaLista';
4
5  function DespesaSaida({despesas, periodo}){
6
7      return (
8          <View>
9              <DespesaSumario despesas={despesas} periodo={periodo}/>
10             <DespesaLista despesas={despesas}/>
11          </View>
12      )
13
14  }
15
16  export default DespesaSaida;
```

Adiciona as
propriedades de
entrada

Atividade

```
import DespesaSaida from '../components/despesa/DespesaSaida'

function TodasDespesas(){

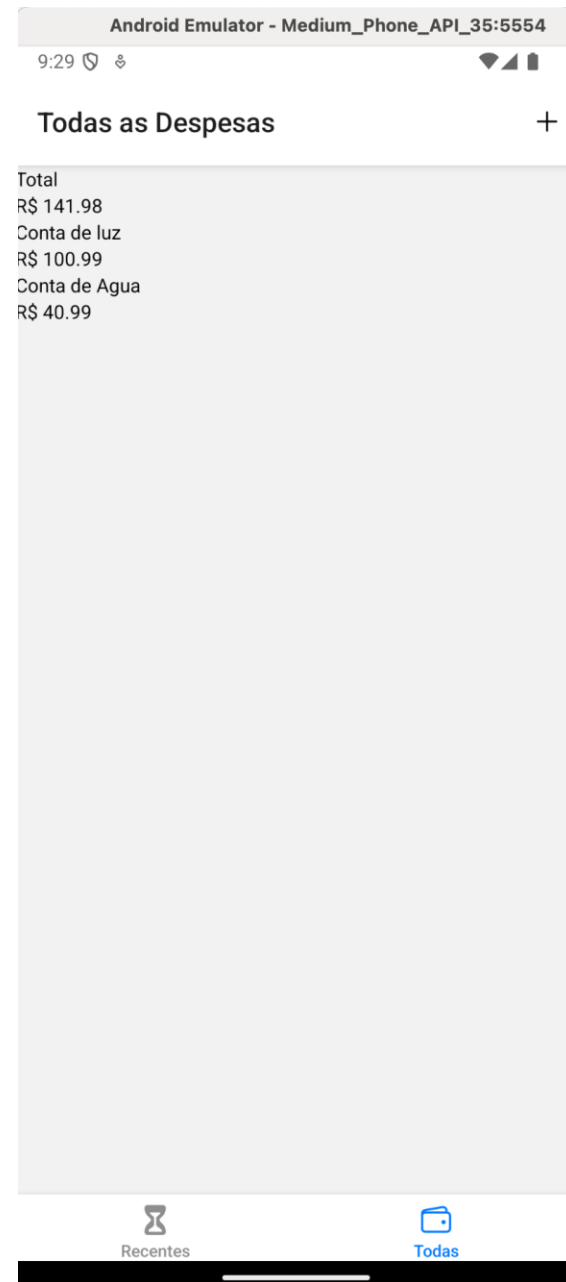
  const DUMMY_DESPESAS = [

    {
      id: '1',
      descricao: 'Conta de luz',
      valor: 100.99,
      data: new Date(2025, 2, 11)
    },
    {
      id: '2',
      descricao: 'Conta de Agua',
      valor: 40.99,
      data: new Date(2025,4,10)
    }
  ]

  return (
    <DespesaSaida despesas={DUMMY_DESPESAS} periodo={'Total'}/>
  )
}

export default TodasDespesas
```

Ajuste o seu código na
tela **TodasDespesas.js**





Despesaltem

- Precisamos melhorar a apresentação dos itens, incluindo a parte de gestão de eventos.
- Para isso, vamos **substituir a função de renderização que criamos em DespesaList** pelo conteúdo da nossa **função principal de Despesaltem**, e vamos construir o componente **Despesaltem**.
- Vamos também adicionar a data da compra na lista de itens.



Date

- Date é um **objeto embutido do JavaScript** que permite **criar, manipular e formatar datas e horas**.
- **Métodos úteis:**

Método	Retorna
.getFullYear()	Ano com 4 dígitos
.getMonth()	Mês (de 0 a 11 → jan = 0, dez = 11)
.getDate()	Dia do mês
.getDay()	Dia da semana (0 = domingo, 6 = sábado)
.getHours()	Hora (0–23)
.getMinutes()	Minutos
.getSeconds()	Segundos

Conversão de Data

- Vamos criar uma função, dentro de **DespesaItem**, para conversão de data para um formato mais legível!

```
function getDataFormatada (data) {  
  return data.getDate() + '/' + (data.getMonth() + 1) + '/' +  
  data.getFullYear();  
}
```

- Agora vamos ajustar o componente Despesa Item!

```
import { View, Text, Pressable, StyleSheet } from 'react-native';

function getDataFormatada(data){
  return data.getDate() + '/' + (data.getMonth() + 1) + '/' + data.getFullYear();
}

function DespesaItem({item}){
  return (
    <Pressable>
      <View style={styles.itemContainer}>
        <View style={styles.itemText}>
          <Text>{getDataFormatada(item.data)}</Text>
        </View>
        <View style={styles.itemText}>
          <Text>{item.descricao}</Text>
        </View>
        <View style={styles.itemText}>
          <Text>R$ {item.valor}</Text>
        </View>
      </View>
    </Pressable>
  );
}
```

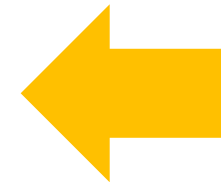
- E importar em DespesaLista!

```
import { View, Text, FlatList } from 'react-native';
import DespesaItem from './DespesaItem';
```

```
function renderDespesaItem(itemData){
  return(
    <View>
      <Text>{itemData.item.descricao}</Text>
      <Text>R$ {itemData.item.valor}</Text>
    </View>
  )
}
```

```
function DespesaLista({despesas}){
  return (
    <FlatList data={despesas} renderItem={DespesaItem}
      keyExtractor={(item) => item.id} />
  );
}
```

```
export default DespesaLista;
```



Atividade

- Crie os estilos CSS de DespesaItem, para que o aspecto da sua aplicação fique semelhante ao apresentado ao lado.

```
function DespesaItem({item}){  
  return (  
    <Pressable>  
      <View style={styles.itemContainer}>  
        <View style={styles.itemText}>  
          <Text>{getDataFormatada(item.data)}</Text>  
        </View>  
        <View style={styles.itemText}>  
          <Text>{item.descricao}</Text>  
        </View>  
        <View style={styles.itemText}>  
          <Text>R$ {item.valor}</Text>  
        </View>  
      </View>  
    </Pressable>  
  );  
}
```

Android Emulator - Medium_Phone_API_35:5554

9:58

Todas as Despesas +

Total		R\$ 141.98
10/3/2025	Conta de luz	R\$ 100.99
20/3/2025	Conta de Agua	R\$ 40.99

Recentes Todas

Atividade

```
const styles = StyleSheet.create({
  itemContainer: {
    flex: 1,
    padding: 5,
    marginVertical: 5,
    marginHorizontal: 5,
    backgroundColor: 'lightgray',
    flexDirection: 'row',
  },
  itemText: {
    flex: 1,
    padding: 2,
    marginVertical: 2,
    marginHorizontal: 2,
    alignContent: 'left',
  },
})
```

Android Emulator - Medium_Phone_API_35:5554

9:58

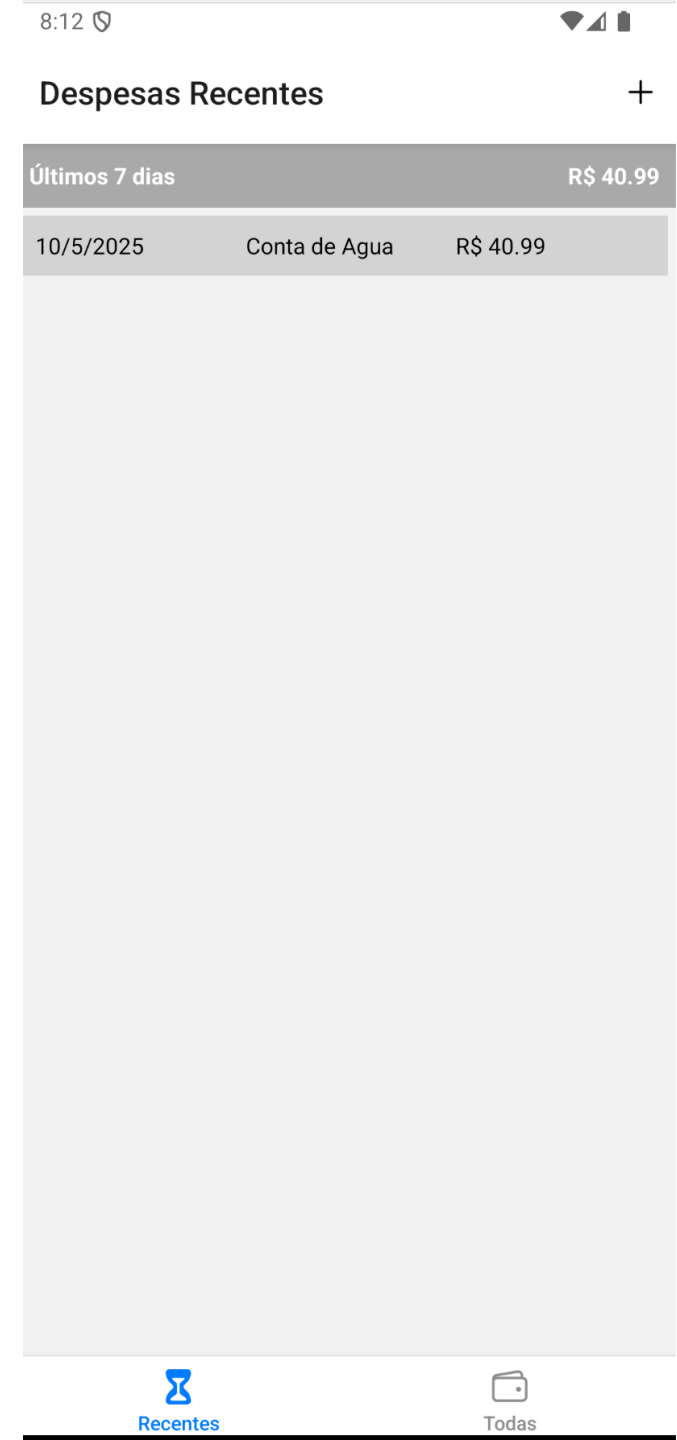
Todas as Despesas +

Total		R\$ 141.98
10/3/2025	Conta de luz	R\$ 100.99
20/3/2025	Conta de Agua	R\$ 40.99

Recentes Todas

Atividade 2

- Crie a tela de Despesas Recentes (DespesaRecentes.js) utilizando os componentes já codificados.
- Ele deve apresentar a despesa dos últimos sete dias.
- Você deve criar uma função para filtrar apenas os últimos sete dias do array de despesas.
- Não deve exibir – “despesas futuras”



Atividade 2

```
import DespesaSaida from '../components/despesa/DespesaSaida';

function DespesaRecentes() {
  function filtrarUltimos7Dias(despesas) {
    const hoje = new Date();
    const seteDiasAtras = new Date();
    seteDiasAtras.setDate(hoje.getDate() - 7);

    return despesas.filter(despesa => {
      return despesa.data >= seteDiasAtras && despesa.data <= hoje;
    });
  }

  const DUMMY_DESPESAS = [
    {
      id: '1',
      descricao: 'Conta de luz',
      valor: 100.99,
      data: new Date(2025, 2, 11)
    },
    {
      id: '2',
      descricao: 'Conta de Agua',
      valor: 40.99,
      data: new Date(2025, 4, 10)
    }
  ]

  return (
    <DespesaSaida despesas={filtrarUltimos7Dias(DUMMY_DESPESAS)} periodo={'Últimos 7 dias'}/>
  )
}
```



Tela Gerenciar Despesas

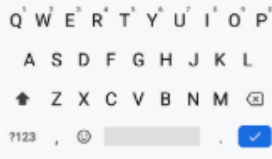
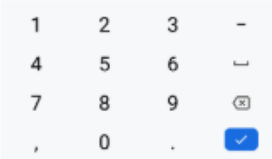
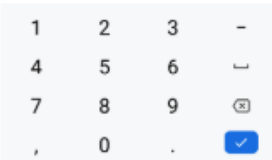
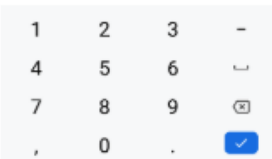
- É a tela onde iremos cadastrar uma nova despesa, editar ou remover uma despesa existente.
- Iremos adicionar componentes do tipo TextInput para entrada de dados, componentes do tipo Text para os rótulos e Button para enviar as alterações!



TextInput

- `maxLength` - Limita o número máximo de caracteres que podem ser inseridos.
- `autoFocus` - Se verdadeiro, concentra a entrada. O valor padrão é falso.
- `editable` - Se falso, o texto não será editável. O valor padrão é verdadeiro.
- `keyboardType` – determina o tipo de teclado a abrir
- <https://reactnative.dev/docs/textinput>

keyboardType

keyboardType	iOS	Android
default		
number-pad		
decimal-pad		
numeric		
email-address		
phone-pad		

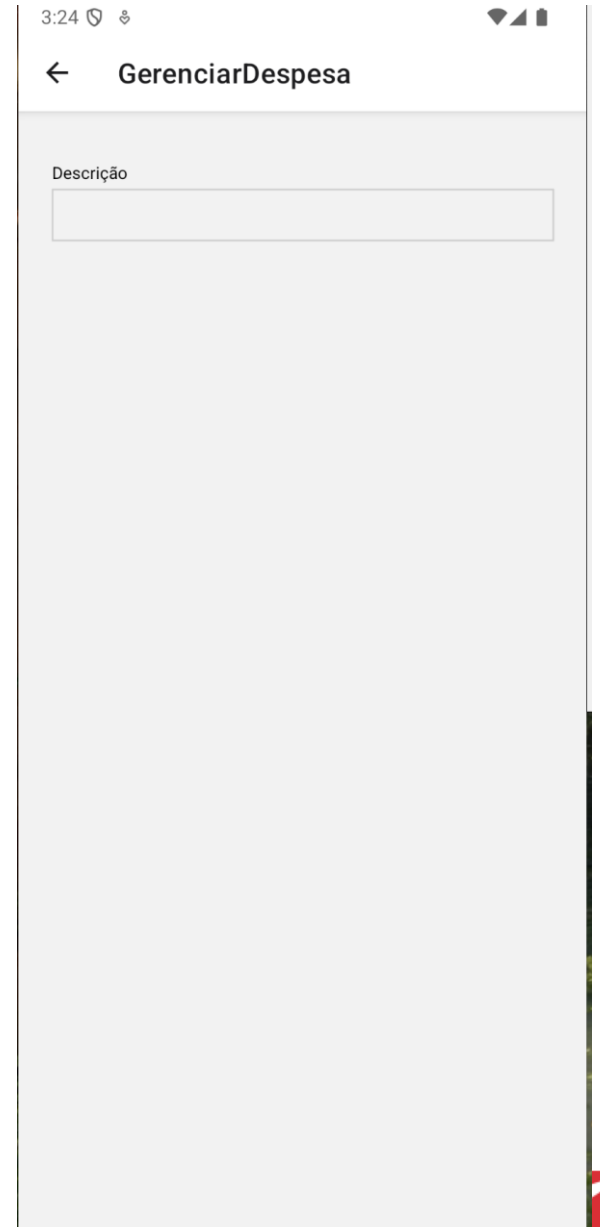
Gerenciar Despesa

```
import {View, Text, TextInput, StyleSheet} from 'react-native';

function GerenciarDespesa() {
  return (
    <View style={styles.container}>
      <View style={styles.inputContainer}>
        <Text style={styles.label}>Descrição</Text>
        <TextInput style={styles.input} maxLength={20}></TextInput>
      </View>
    </View>
  )
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    margin: 20,
  },
  inputContainer: {
    marginHorizontal: 4,
    marginVertical: 16,
  },
  label: {
    fontSize: 12,
    marginBottom: 4,
  },
  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    padding: 8,
  },
});
```

Ajuste o seu código em
GerenciarDespesa.js



The image shows a mobile application interface for managing expenses. At the top, there is a status bar with the time 3:24 and signal icons. Below it is a navigation bar with a back arrow and the title "GerenciarDespesa". The main content area has a light gray background. At the top of this area, the word "Descrição" is displayed above a white rectangular text input field with a thin gray border. The rest of the screen is empty.

Atividade

- Crie as caixas de texto para:
 - valor da despesa
 - e data da despesa.
- A caixa valor da despesa deve abrir um teclado do tipo **decimal-pad**.

3:26



← GerenciarDespesa

Descrição

Valor da Despesa

Data da Despesa

Atividade

```
import {View, Text, TextInput, StyleSheet} from 'react-native';

function GerenciarDespesa() {
  return (
    <View style={styles.container}>
      <View style={styles.inputContainer}>
        <Text style={styles.label}>Descrição</Text>
        <TextInput style={styles.input} maxLength={20}></TextInput>
      </View>

      <View style={styles.inputContainer}>
        <Text style={styles.label}>Valor da Despesa</Text>
        <TextInput style={styles.input}
          keyboardType={'decimal-pad'}></TextInput>
      </View>

      <View style={styles.inputContainer}>
        <Text style={styles.label}>Data da Despesa</Text>
        <TextInput style={styles.input}></TextInput>
      </View>
    </View>
  );
}
```



Atividade

- Crie três constantes, um para cada caixa de texto – data, valor e descrição.
- Use o Hook useState
- Set as funções de alteração – setData, setValor e setDescription.
- Associe cada uma ao TextInput correspondente.

Atividade

```
import {View, Text, TextInput, StyleSheet} from 'react-native';
import React, { useState } from 'react';

function GerenciarDespesa() {

  const [data, setData] = useState('');
  const [valor, setValor] = useState('');
  const [descricao, setDescricao] = useState('');

  return (
    <View style={styles.container}>
      <View style={styles.inputContainer}>
        <Text style={styles.label}>Descrição</Text>
        <TextInput style={styles.input} maxLength={20}
          value = {descricao}></TextInput>
      </View>

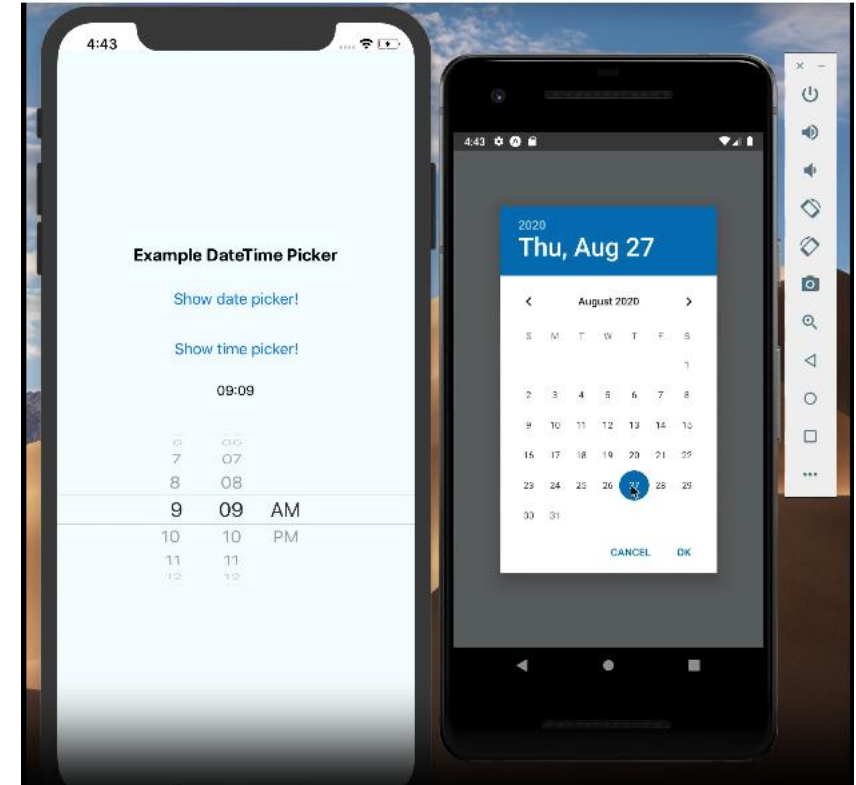
      <View style={styles.inputContainer}>
        <Text style={styles.label}>Valor da Despesa</Text>
        <TextInput style={styles.input}
          keyboardType={'decimal-pad'}
          value = {valor}></TextInput>
      </View>

      <View style={styles.inputContainer}>
        <Text style={styles.label}>Data da Despesa</Text>
        <TextInput style={styles.input}
          value = {data} ></TextInput>
      </View>
    </View>
  )
}
```

datetimepicker

- Vamos agora adicionar o componente de seleção de data.
- Instale o pacote:

`npx expo install @react-native-community/datetimepicker`



Atividade

```
import {View, Text, TextInput, StyleSheet, Pressable} from 'react-native';  
import React, { useState } from 'react';  
import DateTimePicker from '@react-native-community/datetimepicker';
```

```
function GerenciarDespesa() {  
  
  const [data, setData] = useState(new Date());  
  const [valor, setValor] = useState('');  
  const [descricao, setDescricao] = useState('');  
  
  const [showPicker, setShowPicker] = useState(false);  
  const onChange = (event, selectedDate) => {  
    const currentDate = selectedDate || data;  
    setShowPicker(false);  
    setData(currentDate);  
  };  
}
```

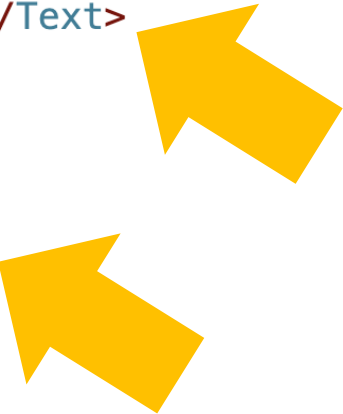
Importe Pressable e
DateTimePicker

Ajuste a constante
data (new Date())

Adicione o código de
showPicker e
onChange

Atividade

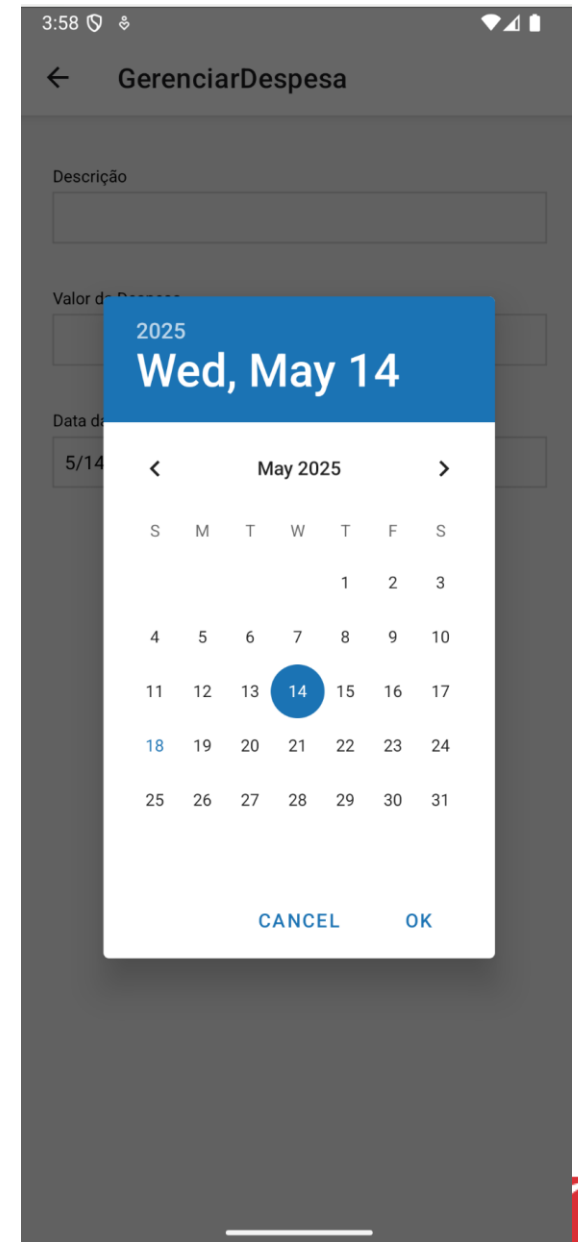
```
<View style={styles.inputContainer}>
<Text style={styles.label}>Data da Despesa</Text>
<Pressable onPress={() => setShowPicker(true)} style={styles.input}>
  <Text>{data.toLocaleDateString('pt-BR')}</Text>
</Pressable>
{showPicker && (
  <DateTimePicker value={data} mode="date"
    display="default" onChange={onChange}
  />
)}
</View>
```



Agora vamos ajustar a entrada de Data da Despesa

Vamos substituir o Input por um texto normal, dentro de Pressable

Vamos configurar DateTimePicker



Vamos validar o valor?

```
const handleChangeValor = (text) => {  
  const cleanText = text.replace(',', '.');  
  
  // Expressão para capturar apenas números com até 2 casas decimais  
  const match = cleanText.match(/^\\d*\\.?\\d{0,2}$/);  
  
  if (match) {  
    setValor(cleanText);  
  }  
};
```

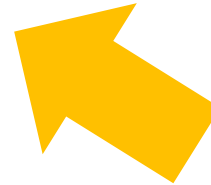
```
<View style={styles.inputContainer}>  
  <Text style={styles.label}>Valor da Despesa</Text>  
  <TextInput style={styles.input}  
    keyboardType={'decimal-pad'} maxLength={10}  
    value = {valor} onChangeText={handleChangeValor}></TextInput>  
</View>
```



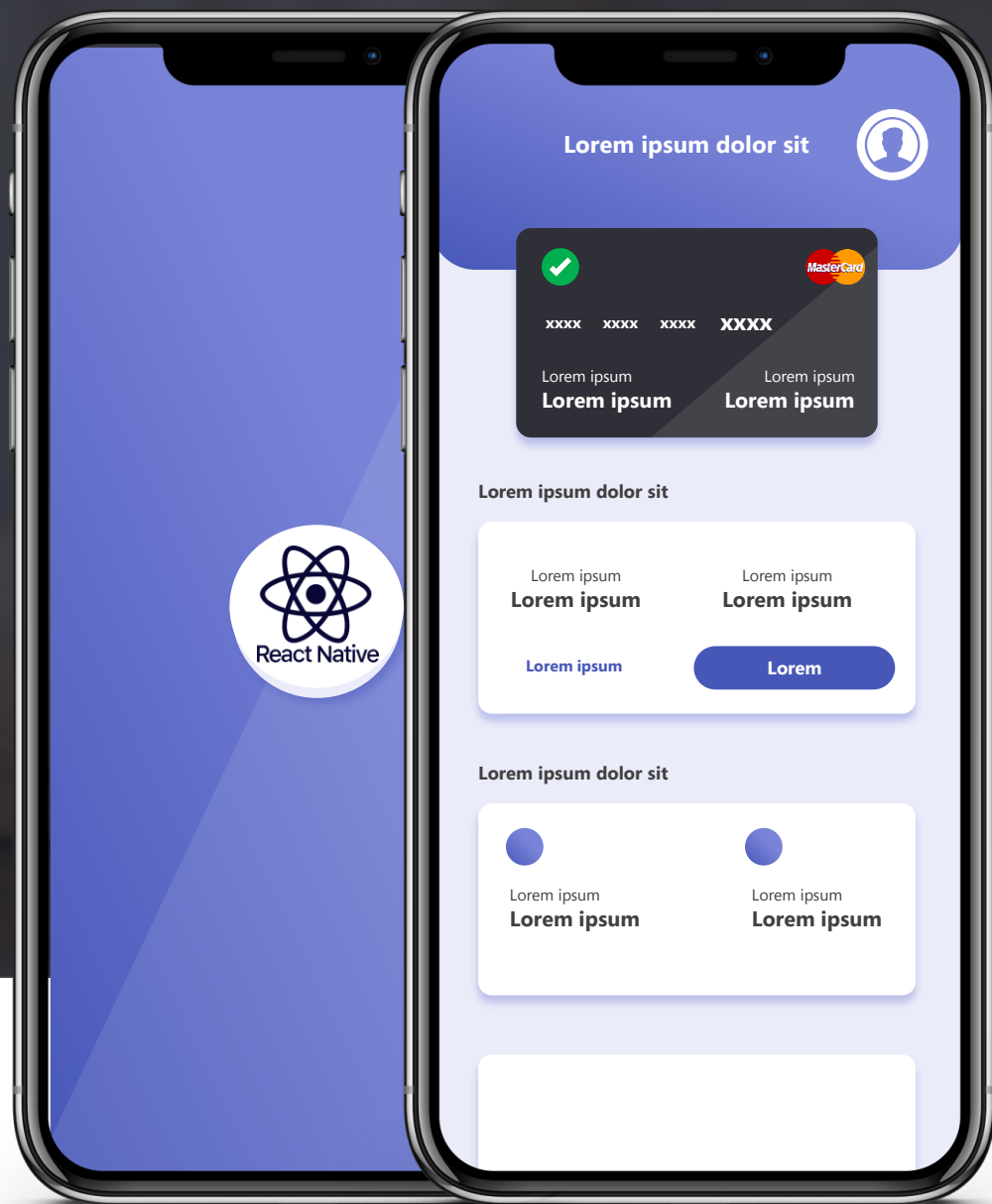


Para terminar...

```
return (  
  <View style={styles.container}>  
    <View style={styles.inputContainer}>  
      <Text style={styles.label}>Descrição</Text>  
      <TextInput style={styles.input} maxLength={20}  
        value = {descricao} onChangeText={setDescricao}></TextInput>  
    </View>  
  </View>  
)
```



Ajuste o componente
de inserção da
descrição, setando
onChangeText



DÚVIDAS